

Python

Python is an object-oriented, high-level, interpreted, general purpose and dynamic programming language.

It was developed by Guido van Rossum in 1991.

The name 'Python' came from an old BBC television comedy sketch series "Monty's Python Flying Circus".

```
In [ ]: #Simple python program  
print('Hello')
```

Hello

```
In [ ]: print('Debashree')
```

Debashree

Keywords : Reserved Words (pre-defined words)

```
In [ ]: #to get python keywords  
help('keywords')
```

Here is a list of the Python keywords. Enter any keyword to get more help.

False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

```
In [ ]:
```

Variable : variables are the names given to the memory location to store a value or it is a container to store a value

We can use upper case letters(A-Z), lower case letters (a-z), digits(0-9)

We can't use special symbols except underscore(_)

We can't use keywords

We can't start with a digit

We can't use space in between the variable name instead we can use underscore.

Python is case sensitive

```
In [ ]: #we can't use keyword as variable name  
for = 10
```

```
Cell In[99], line 2  
    for = 10  
      ^  
SyntaxError: invalid syntax
```

```
In [ ]: #we can't start with a digit  
123var = 20
```

```
Cell In[100], line 2  
    123var = 20  
      ^  
SyntaxError: invalid decimal literal
```

```
In [ ]: #we can't use space in between the variable name instead we can use underscore.  
var 123 = 30
```

```
Cell In[101], line 2  
    var 123 = 30  
      ^  
SyntaxError: invalid syntax
```

```
In [ ]: #we can't use special symbols except underscore(_)  
@!&^% = 40
```

```
Cell In[102], line 2  
    @!&^% = 40  
      ^  
SyntaxError: invalid syntax
```

```
In [ ]: #we can use upper case letters(A-Z), Lower case letters (a-z), digits(0-9)  
VAR = 10  
print(VAR)
```

10

```
In [ ]: var = 20  
print(var)
```

20

```
In [ ]: Var = 30  
print(Var)
```

30

```
In [ ]: var_123 = 40  
print(var_123)
```

40

In []:

In []:

Datatypes

Primary

- **Numeric :**
 - Integer - int
 - Float - float
 - Complex - complex
- **Boolean(bool) :**
 - True
 - False

Sequential

- String - str
- List - list
- Tuple - tuple
- Set - set
- Dictionary - dict

```
In [ ]: #int
a = 10
print(a)
```

10

```
In [ ]: type(a)
```

Out[]: int

```
In [ ]: #float
f = 2.5
print(f)
```

2.5

```
In [ ]: type(f)
```

Out[]: float

```
In [ ]: #complex
c = 2 + 5i
```

Cell In[111], line 2

```
c = 2 + 5i
      ^
```

SyntaxError: invalid decimal literal

```
In [ ]: c = 2 + 5j
        print(c)
```

(2+5j)

```
In [ ]: type(c)
```

Out[]: complex

```
In [ ]: #Boolean
        a = True
        print(a)
```

True

```
In [ ]: type(a)
```

Out[]: bool

```
In [ ]:
```

String (str) : sequence of characters stored inside quotation marks

```
In [43]: s = "Hello Everyone"
        print(s)
```

Hello Everyone

```
In [47]: type(s)
```

Out[47]: str

```
In [49]: a = "abcd1234@$#% 😊 🐼"
        type(a)
```

Out[49]: str

```
In [ ]:
```

```
In [51]: #single line string denoted by ' ' or " "
        a = 'this is a single line'
        b = "this is a single line"

        print(a)
        print(b)
```

this is a single line
this is a single line

```
In [53]: type(a), type(b)
```

Out[53]: (str, str)

```
In [55]: #multi line string is denoted with ''' ''' or """ """  
a = '''this is multi  
line string'''  
b = """this is multi  
line string"""
```

```
In [57]: type(a),type(b)
```

```
Out[57]: (str, str)
```

```
In [ ]:
```

```
In [59]: a = "Tasty Apple"  
print(a)  
type(a)
```

Tasty Apple

```
Out[59]: str
```

```
In [61]: #indexing  
#Syntax : var_name[index_position]  
a[4]
```

```
Out[61]: 'y'
```

```
In [63]: a[-7]
```

```
Out[63]: 'y'
```

```
In [93]: #slicing  
#Forward Slicing  
#syntax = var[start:stop+1:step]  
  
#"sty A"  
a[2:7:1]
```

```
Out[93]: 'sty A'
```

```
In [95]: a[-9:-4:1]
```

```
Out[95]: 'sty A'
```

```
In [97]: #TsyApe  
a[0:11:2]
```

```
Out[97]: 'TsyApe'
```

```
In [99]: a[-11:11:2]
```

```
Out[99]: 'TsyApe'
```

```
In [101... #TtAl  
a[0:10:3]
```

```
Out[101... 'TtAl'
```

```
In [103... a[-11:-1:3]
```

```
Out[103... 'TtAl'
```

```
In [105... a[-11:10:3]
```

```
Out[105... 'TtAl'
```

```
In [107... a[::]
```

```
Out[107... 'Tasty Apple'
```

```
In [109... a[::-1]
```

```
Out[109... 'elppA ytsaT'
```

```
In [111... #Backward Slicing  
#Syntax : var[start_index : stop_index-1 : step]  
  
#"A yts"  
a[6:1:-1]
```

```
Out[111... 'A yts'
```

```
In [113... a[-5:-10:-1]
```

```
Out[113... 'A yts'
```

```
In [ ]:
```

String Functions

`capitalize()` : converts the first character to upper case and the rest lower case.

`upper()` : converts all the characters into upper case

`lower()` : converts all the characters into lower case

`casefold()` : converts all the characters into lower case

`center()` : returns a centered string

`count()` : it counts how many times a particular character is repeated

`index()` : returns index position of a particular character

`strip()` : removes spaces present before and after the string

```
In [116... a = "heLLlO eVeRYoNE"  
print(a)
```



```
Out[150...] ['ram', 'sita', 'gita']
```

```
In [152...] 'ram_sita_gita'.split('a')
```

```
Out[152...] ['r', 'm_sit', '_git', '']
```

```
In [154...] '          hello          '.strip()
```

```
Out[154...] 'hello'
```

```
In [158...] '*****hello*****'.strip('*')
```

```
Out[158...] 'hello'
```

```
In [162...] '*****%*%*%*%*%hello*****%*%*%*%*%'.strip('%*')
```

```
Out[162...] 'hello'
```

```
In [166...] '          hello          '.lstrip()
```

```
Out[166...] 'hello          '
```

```
In [168...] '          hello          '.rstrip()
```

```
Out[168...] '          hello'
```

```
In [170...] 'abc'.isalpha()
```

```
Out[170...] True
```

```
In [172...] '123'.isnumeric()
```

```
Out[172...] True
```

```
In [174...] '123'.isdigit()
```

```
Out[174...] True
```

```
In [176...] 'abc123'.isalnum()
```

```
Out[176...] True
```

```
In [178...] 'The Fairy Tale'.istitle()
```

```
Out[178...] True
```

```
In [180...] 'var 123'.isidentifier()
```

```
Out[180...] False
```

```
In [182...] 'Var_123'.isidentifier()
```

```
Out[182...] True
```

```
In [ ]:
```


Calendar

In [186...

```
import calendar
print(calendar.calendar(2025))
```

2025																				
January							February							March						
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su
			1	2	3	4							1	2					1	2
6	7	8	9	10	11	12	3	4	5	6	7	8	9	3	4	5	6	7	8	9
13	14	15	16	17	18	19	10	11	12	13	14	15	16	10	11	12	13	14	15	16
20	21	22	23	24	25	26	17	18	19	20	21	22	23	17	18	19	20	21	22	23
27	28	29	30	31			24	25	26	27	28			24	25	26	27	28	29	30
														31						
April							May							June						
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su
		1	2	3	4	5				1	2	3	4							1
7	8	9	10	11	12	13	5	6	7	8	9	10	11	2	3	4	5	6	7	8
14	15	16	17	18	19	20	12	13	14	15	16	17	18	9	10	11	12	13	14	15
21	22	23	24	25	26	27	19	20	21	22	23	24	25	16	17	18	19	20	21	22
28	29	30					26	27	28	29	30	31		23	24	25	26	27	28	29
														30						
July							August							September						
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su
		1	2	3	4	5					1	2	3	1	2	3	4	5	6	7
7	8	9	10	11	12	13	4	5	6	7	8	9	10	8	9	10	11	12	13	14
14	15	16	17	18	19	20	11	12	13	14	15	16	17	15	16	17	18	19	20	21
21	22	23	24	25	26	27	18	19	20	21	22	23	24	22	23	24	25	26	27	28
28	29	30	31				25	26	27	28	29	30	31	29	30					
October							November							December						
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su
			1	2	3	4							1	1	2	3	4	5	6	7
6	7	8	9	10	11	12	3	4	5	6	7	8	9	8	9	10	11	12	13	14
13	14	15	16	17	18	19	10	11	12	13	14	15	16	15	16	17	18	19	20	21
20	21	22	23	24	25	26	17	18	19	20	21	22	23	22	23	24	25	26	27	28
27	28	29	30	31			24	25	26	27	28	29	30	29	30	31				

In [192...

```
print(calendar.month(2025,11))
```

November 2025						
Mo	Tu	We	Th	Fr	Sa	Su
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30

In []:

LIST (list)

Sequence of items denoted with '[]' and separated with ','.

It is ordered(indexed with specific position), mutable(items can be changed) and heterogenous(supports multiple data-type).

```
In [194... l = [20,2.5,'cat',True]
print(l)
```

```
[20, 2.5, 'cat', True]
```

```
In [196... type(l)
```

```
Out[196... list
```

```
In [198... #indexing syntax : var[index]
l[3]
```

```
Out[198... True
```

```
In [200... l[-1]
```

```
Out[200... True
```

```
In [ ]:
```

```
In [202... #slicing
l[0:3:2]
```

```
Out[202... [20, 'cat']
```

```
In [204... l[::]
```

```
Out[204... [20, 2.5, 'cat', True]
```

```
In [206... l[::-1]
```

```
Out[206... [True, 'cat', 2.5, 20]
```

```
In [ ]:
```

List Methods

Adding Items to a List

- `var.append(obj)` : add item to the end of the list
- `var.insert(ind,obj)` : add item to the given index of the list
- `var.extend([sequence])` : add items from the given sequence at the end of the list

Removing Items from the List

- `var.remove(obj)` : removes the first matched instance of object from the list
- `var.pop(index)` : it pop or delete the item of the given index from the list

- `var.clear()` : empty the list

Miscellaneous methods

- `var.copy()` : copy the item
- `var.count(obj)`
- `var.index(obj)`
- `var.sort()` : sort or arrange in ascending order
- `var.sort(reverse=True)` : arrange in descending order
- `var.reverse()` or `var[::-1]` can be use to reverse the list

```
In [208...  l = [25,60,35,45,35,98]
            print(l)
```

```
[25, 60, 35, 45, 35, 98]
```

```
In [210...  #adding items to a list
            #append
            print(l)
            l.append(100)
            print(l)
```

```
[25, 60, 35, 45, 35, 98]
[25, 60, 35, 45, 35, 98, 100]
```

```
In [212...  #insert
            print(l)
            l.insert(2,55)
            print(l)
```

```
[25, 60, 35, 45, 35, 98, 100]
[25, 60, 55, 35, 45, 35, 98, 100]
```

```
In [214...  #extend
            print(l)
            l.extend([10,20,30])
            print(l)
```

```
[25, 60, 55, 35, 45, 35, 98, 100]
[25, 60, 55, 35, 45, 35, 98, 100, 10, 20, 30]
```

```
In [ ]:
```

```
In [216...  #removing elements from a list
            #pop
            print(l)
            l.pop()
            print(l)
```

```
[25, 60, 55, 35, 45, 35, 98, 100, 10, 20, 30]
[25, 60, 55, 35, 45, 35, 98, 100, 10, 20]
```

```
In [218...  print(l)
            l.pop(2)
            print(l)
```

```
[25, 60, 55, 35, 45, 35, 98, 100, 10, 20]
[25, 60, 35, 45, 35, 98, 100, 10, 20]
```

In [220]...

```
#remove  
print(l)  
l.remove(35)  
print(l)
```

```
[25, 60, 35, 45, 35, 98, 100, 10, 20]  
[25, 60, 45, 35, 98, 100, 10, 20]
```

In [222]...

```
#clear  
print(l)  
l.clear()  
print(l)
```

```
[25, 60, 45, 35, 98, 100, 10, 20]  
[]
```

In []:

In [30]:

```
#Miscellaneous methods in list  
#copy
```

In [32]:

```
#count
```

In [34]:

```
#index
```

In [36]:

```
#sort - ascending order
```

In []:

In [39]:

```
#reverse
```

In [41]:

```
#clear
```

In []:

Tuple (tuple)

An ordered but immutable(which cannot change size or permanent) collection of items.

It is denoted by parenthesis '()' and separated by ','.

In []:

In []:

In []:

```
#indexing
```

In []:

```
#slicing
```

Tuple Methods